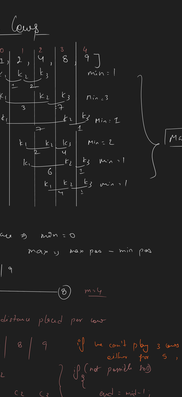
Aggressive Cows

stalls[] = [1, 2, 4, 8, 9]

k = 3

Output = 3

All permutations ⇒



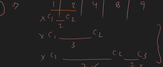
① linear ⇒ $O(n^2)$

② B.S ⇒ Search space ⇒ min = 0

max ⇒ max pos − min pos

Cow = 3 → at least 4 distance placed per cow

① ⇒

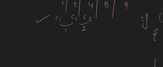


if we can't place 3 cows at distance of 4 then mostly, we can't either for 3, 2, 1 ... cow

if (not possible ret)

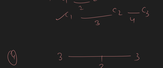
ans = mid−1;

②



if (possible)
{ ans = mid;
start = mid+1;
}

③



if (possible)
{ ans = mid;
start = mid+1;
}

④

if (possible)
{ ans = mid;
start = mid;
}

(start = 4 > end = 3) ← terminate

isPossible Sol (arr, mid, k) k = 3 mid = 4

⇒ {

0	1	2	3	4
1	2	4	8	9
k_1		k_2	k_3	

p = 0 < arr[0]
0 < 1

p = arr[0] = 1

p = arr[1]
abs(−)
(1 − mid) ≥ 4

p = arr[2] = 1 − 4 = 3 > mid
p = arr[3] = 1 − 8 = 7 > mid → p = arr[mid] = 8 | k − −;

p = arr[4] = 8 − 9 > mid

arr E.O.L()
return false

⇒

0	1	2	3	4
1	2	4	8	9
p				

k = 3 / mid = 4
P = arr[0]

p = 1

abs(p − arr[2])
1 − 1 = 0 ≥ mid) ×
{ inc 3

abs(p − arr[3])
1 − 2 = (1 ≥ 4) ×
{ (inc)}

(p − arr[4])
1 − 4 = (3 ≥ 4) ×
{ inc 4

(p − arr[5])
1 − 8 = (7 ≥ 4) ✓ k = 2
} k − −;
p = arr[5] = 8
}

(p − arr[6])
8 − 9 = (1 ≥ 4) ×
{ inc → (i++)
}
return false
}

Trace Sol

0	1	2	3	4
1	2	4	8	9
P	P			

k = 3 mid = 4

p = arr[0] = 1

(p − arr[1])
1 − 1 = (0 ≥ 1) ×
{ inc }

(p − arr[2])
1 − 4 = (1 ≥ 1) ✓
{ k − −; k = 2
p = arr[2] = p = 4

(p − arr[3])
4 − 8 = (4 ≥ 1) ✓ terminate return true;
{ k − −; k = 1
p = arr[3] = p = 8

0	1	2	3	4
1	2	4	8	9
1	1	1	1	

k = 3
mid = 4

p = arr[0] = 1
i = 0 ✗ ✗ ✗ ✗ 5

while (i < n) 0 < 5 | 1 < 5 | 2 < 5 | 3 < 5 | 4 < 5 | 5 < 5 X

{ if (abs(p − arr[i]) ≥ mid) 1 − 1 = 0 ≥ 4 ✗ | 1 − 2 = 1 ≥ 4 ✗ | 1 − 4 = 3 ≥ 4 ✗ | 1 − 8 = 7 ≥ 4 ✓ | 8 − 9 = 1 ≥ 4 ✗
{
k − −; 2
p = arr[i], p = 8
}
if (k ≤ 1) 3 ≤ 1 ✗ | 3 ≤ 1 ✗ | 2 ≤ 1 ✗ | 2 ≤ 1 ✗ | 2 ≤ 1 ✗
{ return true;
}
i++;
}
return false; ✓✓

0	1	2	3	4
1	2	4	8	9

k = 3
mid = 4

p = arr[0] = 1
i = 0 ✗ 2
while (i < n) 0 < 5 | 1 < 5 | 2 < 5
{ if (abs (p − arr[i]) ≥ mid) 1 − 1 = 0 ≥ 4 ✗ | 1 − 2 = 1 ≥ 4 ✗ | 1 − 4 = 3 ≥ 4 ✓
{ k − −; ✗ 2
p = arr[i]; p = 2 | p = 4
}
if (k ≤ 1) 2 ≤ 1 ✗ | 2 ≤ 1 ✗ | 1 ≤ 1 ✓
{ return true;
}
i++;
}
return false;